2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD) | 979-8-3315-1560-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCAD66269.2025.11240874

CIMTester: An Agile Golden-Result-Free BIST Compiler for Robust Compute-in-Memory

Wenjie Ren[1], Meng Wu[1], Mingxuan Li[1], Peiyu Chen[2], Tianyu Jia[1,*] and Le Ye[1,*]
[1]Peking University, Beijing, China; [2]Advanced Institute of Informative Technology of Peking University, Hangzhou, China;
[*]Corresponding author email: {tianyuj, yele}@pku.edu.cn

***Abstract*—Digital compute-in-memory (DCIM) is playing an increasingly vital role in efficient AI computing due to its significant efficiency advantages. However, the combination of memory and computation logics in DCIM presents challenges for testing, including extra coupling fault between memory and logic, high overhead for golden result generation and indirect fault location. In this paper, we present a golden-result-free BIST structure together with a computation-coupled CIM BIST algorithm to reduce testing overhead and improve test coverage. In addition, we present CIMTester, a DCIM BIST compiler to adapt to swiftly changing test requirements and DCIM macro sizes and numbers in chips. The template-based generator generates BIST RTL based on proposed BIST structure and modifies the templates according to the architecture parameters of test chip. CIMTester's iterator analyzes diverse sharing strategy of BIST components to satisfy user specifications of area, test time and fault coverage. We implemented and evaluated a series of TSMC 22nm DCIM macros with the generated BIST circuits, which achieves up to 99.48% fault coverage with only less than 2.44% area overhead.**

***Index Terms*—Digital CIM, Testing, BIST Compiler**

I. Introduction

The development of deep neural networks (DNNs) has exacerbated the conventional "memory wall" problem in computing hardware. Compute-in-memory (CIM) is a promising technique that brings computation capability into memory to reduce the data movement cost and power consumption. Due to the advantages of compatibility with advanced technology nodes, SRAM-based CIM has drawn significant attention from industry and academia. For example, TSMC presented SRAM-based CIM using the most advanced 5nm, 4nm, 3nm in the recent three years [1], [2], [3].

Among diverse SRAM CIM architectures, digital CIM (DCIM) has become the major design choice due to high PVT robustness, bit precision and technology scaling ability. Given the extensive deployment of DCIM macros in AI chips [4], [5], [6], [7], they are expected to be robust and have self-test ability to ensure the system level reliability. Moreover, with technology scaling down and design complexity continuously growing, the chip failure probability will also increase. Hence the reliability and testability of DCIM should be carefully guaranteed before high-volume manufacturing.

For conventional memory IPs, built-in self-test (BIST) is an effective and swift way to verify their functionality and enhance their reliability in chip design. On-chip BIST enables autonomous and efficient self-testing, reducing dependency on expensive automatic test equipment (ATE). BIST has been widely adopted in SRAM memory with well-studied memory BIST (MBIST) algorithms, such as MSCAN, Checkerboard, GALPAT, and March [8], [9]. As demonstrated in Fig.1, test vectors for SRAM are generated and sequentially sent into the device under test (DUT) in accordance with BIST algorithms. In MBIST, the test vector space typically consists of simple vectors. This space remains relatively small and independent of scale.

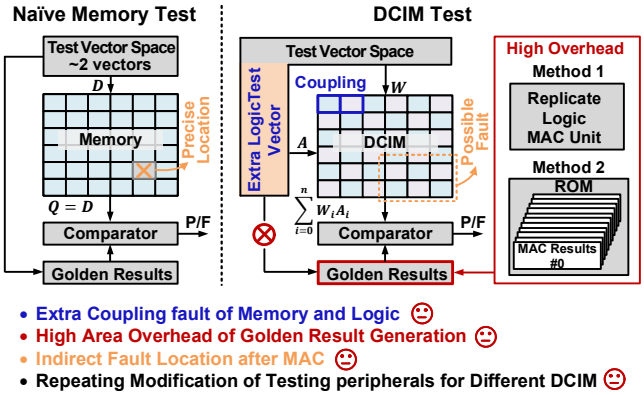


Fig. 1. Conventional memory testing, and the testing challenges for CIMs.

However, current MBIST schemes fail to accommodate DCIM testing, which necessitates a thorough understanding of both DCIM circuit and DFT fundamentals. First, extra fault analysis and test vector generation are required for coupled memory and MAC logic. As the bit-cell with MAC function in DCIM is different from standard memory, the memory fault models and MBIST test schemes are not enough for DCIM. Second, the generation of golden test results for MAC operations in DCIM can be costly. As shown in Fig. 1, the comparator in BIST structures receives the DUT output and the golden reference result to diagnose the DUT. For DCIM testing, a replicate logic MAC unit or vector storage ROM is required, which incurs large area overhead. Finally, the faults in DCIM are difficult to locate after the MAC operation through direct comparison with inputs. The many-to-one mapping from DCIM inputs to computation outputs leads to significant challenges in precise fault location.

Recently, many testing scheme for DCIM with simple logic were reported. [10], [11], [12] introduced testing techniques for 8T-SRAM CIM. March-C-8 testing algorithm is proposed

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:48 UTC from IEEE Xplore. Restrictions apply.

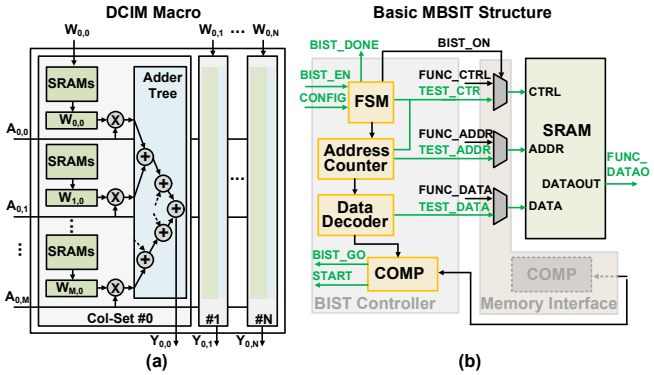


Fig. 2. The basic structure of DCIM and MBIST.

in [10] to cover the typical functional faults and process variation-induced faults. The reliability issues and design-for-testability of DCIM are discussed in [13], [14]. DCIM can operates in computing modes and memory modes. This different behavior of CIM in two modes may cause some computation specific faults. Therefore, fault analysis and CIM testing must be performed in both modes. Prior study [15] have developed testing circuits for CIMs with MAC function. However, computation coupled faults is not analyzed and achieves relatively low test coverage. [16] proposed a BIST strategy for measuring the inaccuracies in analog CIM with MAC operation; however, this approach is not suitable for DCIM. Therefore, there is a notable gap between wide deployment requirements and an efficient test scheme for DCIM.

To solve the above issues, in this work, we present an efficient BIST structure for robust DCIM macro with negligible area overhead. A BIST compiler called CIMTester is developed to automate the BIST circuit generation for multi-CIM systems. First, fault analysis of DCIM is performed on both computing circuits and memory array. We present a computation-coupled BIST algorithm based on March C- to enhance fault coverage of computing related faults. Meanwhile, at the circuit level, a self-comparable BIST structure is introduced to eliminate the overhead of golden result generation, with which the fault location can be easily recognized at the column level. To automate the generation of BIST circuits for DCIM macros, we develop a BIST compiler which integrate a fault simulator and template-based RTL generator to explore the design space of BIST architecture in multi-CIM system under user specification. The generated BIST circuit for a series of TSMC 22nm DCIM macros achieves fault coverage of up to 99.48%, with an area overhead of less than 2.44%.

The contributions of this work are summarized as follows:

- We introduce CIMTester, a CIM BIST compiler that integrates a fault simulator and template-based RTL generator to explore the design space of CIM BIST architecture.
- The fault analysis is performed on DCIM modes, with a computation-coupled BIST algorithm presented to enhance test coverage and efficiency.
- Golden-result-free BIST structure is presented for DCIM to reduce extra area overhead, and a self-comparable circuit is designed to precisely locate faults in CIM.

TABLE I
REPRESENTATIVE MBIST ALGORITHMS.

Test Algorithm	Complexity	Operation	Fault Detected
Zero-One	4N	⇳(w0);⇳(r0);⇳(w1);⇳(r1)[a]	SAF
MATS++	6N	⇳(w0);⇑(r0,w1);⇓(r1,w0,r0)	SAF/TF
March Y	8N	⇳(w0);⇑(r0,w1,r1); ⇓(r1,w0,r0);⇳(r0);	SAF/TF/CF[b]
March C-	10N	⇳(w0);⇑(r0,w1);⇑(r1,w0); ⇓(r0,w1);⇓(r1,w0);⇳(r0)	SAF/TF/CF[b]
March SR	14N	⇳(w0);⇑(r0,w1,r1,w0);⇑(r0,r0); ⇑(w1);⇓(r1,w0,r0,w1);⇓(r1,w1);	SAF/TF/CF[b]

[a]⇑, ⇓, ⇳ stand for address changing in ascending order, descending and either. w0 and r0 mean write 0 and read 0.
[b]Partially coverage

- Our methodology is validated using a series of DCIM macros, achieving superior up to 99.48% fault coverage with low overhead.

II. BACKGROUND

A. Fault Models and Fault Coverage

A fault is the representation of a manufacturing defect in an abstracted function level, which will produce incorrect "error" outputs. The faults in memory arrays include stuck-at-fault (SAF), transition fault (TF) and coupling fault (CF) [8]. SAF is the fault with a fixed 0/1 value for a particular cell and CF is the case when the changing of a cell value alters the storage of another cell. In logic circuits, the prevalent faults are SAF, stuck-open fault (SOF), TF, and path-delay faults (PDF) [17]. TF and PDF are timing-related faults, indicating a switch failure from 0 to 1 or vice versa and violation of path delay.

In digital logic fault diagnosis, fault models are utilized to simulate test patterns and evaluate fault coverage. For example, various faults are initially added to a fault list. Then, a fault is injected into the fault model of the circuit under test. The test patterns are applied to the fault-injected circuit model to output expected error results. This process is repeated until the fault list is exhausted. The fault coverage (FC) for a given set of test vectors is defined as the percentage of faults detected by test vectors [18]. In practice, 100% fault coverage is almost impossible due to the presence of undetectable faults. These metrics are critical to estimating defective parts per million (DPPM), which is directly linked to yield in manufacturing.

Faults in the DCIM macro can be categorized into two types: faults in SRAMs and faults in computing logic. For memory faults, conventional MBIST algorithms utilizing test vectors of 0 and 1 are effective and comprehensive. However, additional test patterns are necessary to address the faults in computational logic. To evaluate a set of generated test patterns, a fault model for DCIM computing logic is required.

B. DCIM Architecture and Memory BIST

As shown in Fig.2 (a), DCIM has an array structure similar to SRAM while computation logic is inserted near bit cells. Fig. 2 shows the blcok diagram of a DCIM macro with M rows and N col-sets. A DCIM macro consists of of multiple col-sets. A col-sets is composed of SRAMs with bit-multiplication (BM) logic and an adder tree [1], [2], [19]. The weights

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:48 UTC from IEEE Xplore. Restrictions apply.

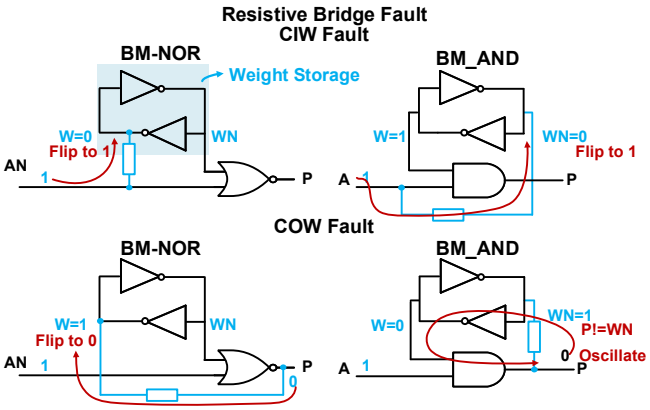


Fig. 3. Resistance bridge faults between SRAM cell and bit-multiplication logic in BM-NOR and BM-AND CIM structures

are stored in SRAM, while the activations are broadcast to multiply with the weights at each column, being accumulated by afterward adder trees. Computation operation activates multiple word lines to retrieve weights from multiple rows of SRAMs, in contrast to the single word line activation in standard memory read operations. These weights are directly input into computation logic without any intuitive observable ports, which makes DCIM testing a more challenging problem.

A typical MBIST structure for memory is shown in Fig.2 (b), which consists of a BIST finite state machine (FSM), address counter, data decoder, comparator and mux interface. The pattern generation of MBIST is related to the MBIST algorithms. Table I lists a few representative MBIST algorithms and their complexities. For example, March C- is a classic MBIST algorithm with a complexity of $10N$, which detects all SAFs, TFs and partial CFs in SRAMs [8].

The operations are described using three types of notations: address sequence changes ($\Updownarrow/\Uparrow/\Downarrow$), test patterns ($0/1$) and memory operations (read/write). From the operation notation of March C-, it can be derived as follows:

- **Step 1:** all memory cells will be wrote to 0s, regardless of the order of the address.
- **Step 2:** a read operation (expected output 0) and then a "write 1" operation are performed on all cells in address sequence changes in ascending order.
- **Step 3-10:** Similar read/write operations of 0/1 in ascending/descending order are performed on all SRAM cells.

It can be observed that the test patterns in MBIST algorithms are simple (all 0 or all 1) and independent of array size due to the symmetric and repetitive structure. However, the adder tree in DCIM introduces carry operation between columns and data coupling between rows (displayed in Fig.5), reducing its symmetry properties. In this context, conventional MBIST algorithms demonstrate low test coverage when applied to DCIM testing.

III. Fault Analysis in DCIM Macro

Analog CIM can achieve high energy efficiency, but shows low accuracy since it is sensitive to the variation of PVT. Prior work shows digital CIMs can achieve high energy efficiency without sacrificing computation accuracy [1], [2], [3], [4]. Therefore, this paper focus on the fault analysis and testing of DCIM with MAC function. DCIM mainly consists of SRAM arrays, BM circuits and adder trees. Since adder tree can be considered a random logic circuit, existing fault models of digital circuit can be used for its testing. This section focus on the fault analysis in BM circuit. BM circuit in CIM array directly coupled the weight and activation, which induces extra potential defects. As shown in Fig. 3, BM circuit consists of a BM gate and a local latch that stores weight from subarray [20] (this latch can also represent the SRAM cell in CIM without read/write path [1]). There are two basic ways to realize the BM logic of $P = AW$ (BM-AND or BM-NOR). The most straightforward is to use an AND gate (BM-AND). Some works use NOR gates (BM-NOR) instead of AND gates to minimize the area, achieving BM logic by $P = (A' + W')' = AW$ through the inversion of inputs and weights [19]. Similar to conventional digital circuits, defects within the AND (NOR) gates can be modeled as stuck-at and delay faults at inputs and outputs. For the defects coupling BM logic and latch, only resistive bridge defects are analyzed since the other defects cannot affect two nodes/lines [21]. Without loss of generality, we perform the fault analysis for the defects between BM gate and latch cell in both BM implementation.

A. Coupling Fault of Input and Weight

Fig. 3 top shows the resistive bridge defects between the input activation (A/AN) and latch node (W/WN) in BM-NOR and BM-AND cases. In DCIM, the driving capability of the input buffer for activation broadcasting is larger than the inverters of the latch cell [21]. As illustrated in Fig. 3, a fault between node AN (A) and W (WN) in the BM-NOR (BM-AND) case causes the stored latch data to flip if AN (A) and W (WN) assume opposing logic states. Conversely, the defect remains benign when A matches the logical complement of W. A comparable effect occurs if a bridge defect arises directly between A and W. To activate the coupling fault of input and weight (CIW), bit-multiplication of A and W should be performed when writing 0 or 1 to latch cells. Subsequent read operation and output validation is performed to observe the fault effect.

B. Coupling Fault of Output and Weight

As illustrated in the bottom of Fig. 3, a resistive bridge defect between the output and the latch cell induces coupling fault between the output and weight (COW). In BM-NOR configuration, if AN is 1, the output P will always be 0. The stored data in latch will be flipped only when W is 1. When AN is 0, the output P is equal to W, inducing no fault. However, in BM-AND case, the COW faults might cause the node P oscillate. As shown in Fig. 3, if A=1 and W=0, then P will be 0 which forces WN (1) to 0 (P!=WN). Then, P will change to 1 and force WN (0) to 1. The output node will oscillate between 0 and 1. On the other hand, if A is 0, output P is 0 regardless of W is 0 or 1. Then, cell value might be flipped when WN is 1. Similar deduction can be

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:48 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 March CCC algorithm

1: **Input: MBIST algorithms (March C-), target coverage (FC_t), ATPG vectors (W, A)**
2: **Output: CIM-BIST pattern sequencer**
3: Initialize March C- sequencer
4: **Step 1:** ⇳(w0); ⇑(r0)
5: **Step 2:** ⇑(w1, A1, r1)
6: **Step 3:** ⇑(w0, A0, r0); ⇓(r0)
7: **Step 4:** ⇓(w1, A0, r1)
8: **Step 5:** ⇓(w0, A1, r0); ⇑(r0)
9: **for** 1 ≤ i ≤length(ATPG vectors) **do**
10: **for** 1 ≤ j ≤ich **do**
11: Write $W_{i,j} = Test_W_{i,j}$
12: **end for**
13: Set activation $A_{i,j} = Test_A_{i,j}$
14: **Computing:** $Y_i = \sum_{j=0}^{ich} A_{i,j} \times W_{i,j}$
15: **Evaluation step:** $FC = \frac{Detecte\ fault}{All\ fault}$
16: **if** $FC < FC_t$ **then**
17: i=i+1
18: **else**
19: Exit
20: **end if**
21: **end for**
22: **return** March CCC sequencer

performed when the bridge defects exists between output P and W nodes. For the BM-NOR case, the flip of data can be verified by memory reading operations. For the BM-NAND case, the oscillation behavior causes an intermittent fault [21]. The periodic switching of output node P or latch cell can be observed by multiple read operations or the variation of current I. According to the discussion above, the activation of COW faults in BM-NOR require AN=1 and W=1 while the activation of fault in BM-AND case requires A=1 or A=0 and WN=1. Read operation is followed to observe the fault effect.

C. Computation Coupled BIST Algorithm for DCIM

According to above discussion, the testing steps of DCIM can be divided into the testing of SRAM array, BM logic and the adder tree. The BIST algorithm for DCIM should cover the conventional faults in SRAM array, address decoder and peripheral read/write circuit. It is better to develop the DCIM BIST algorithm based on widely used MBIST algorithm. The adder tree can be seen as a regular random logic circuit. Therefore, we can use commercial automatic test pattern generation (ATPG) tools to generate efficient test patterns for the adder tree under given fault models.

Algorithm 1 formally outlines an example of computation coupled BIST algorithm based on March C-. March C- algorithm (listed in Table I) are widely used for memory fault detection due to their linear time complexity. The inputs of Algorithm 1 are a selected MBIST algorithm (in this case, March C-), the target fault coverage (FC_t), and additional test vectors generated by ATPG for adder tree. March C- test (line 4) starts with ⇳(w0) and ⇑(r0). An additional A1 operation is inserted in step 2 of March C- test (line 5), where A1 denotes to set DCIM to computing mode and activation to 1. This step can cover the CIW faults in both BM-AND and BM-NOR

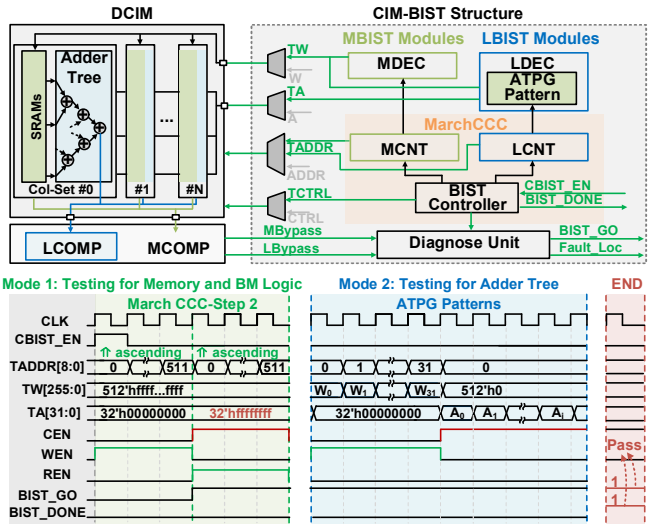


Fig. 4. Proposed golden-result-free BIST structure for DCIM and the BIST controller waveform for March CCC algorithms.

structures. The following step 3 inserts A0 before two read operations because it is used to detect COW faults (oscillating) in BM-AND and also CIW faults in both structures (line 6). Step 4 and 5 performs bit-multiplication of A0W1 and A1W0 to cover the COW faults in BM-NOR and COW faults in BM-AND separately (line 7-8). Extra computing tests for adder tree are added according to FC_t (line 9-20). To apply a ATPG vector, ich (input channel) write operations are performed to load all weights to SRAM cells and one computing operation is required with corresponding A_i. Once the target FC is achieved, the final sequencer representing the step-by-step BIST flow is returned (line 22) as the used algorithms in subsequent BIST RTL generation.

IV. Proposed BIST Strategies

A. BIST Structure Overview

In this section, we introduce the design details of our golden-result-free CIM-BIST and the waveform of BIST controller implementing March CCC algorithm. As demonstrated in Fig. 4 (top), the CIM-BIST structure comprises two sets of circuits: MBIST for memory and coupled LBIST for computation logic. Each set consists of a data decoder (MDEC/LDEC), an address counter (MCNT/LCNT) and a comparator (MCOMP/LCOMP). These modules are all controlled by a centralized BIST controller to execute the March CCC algorithm efficiently. The whole system can operation in two modes: testing mode and operation mode. In testing mode, the computing and memory ports and enable signals are controlled by BIST controller, while DCIM is controlled by user and can be used normally as IP in operation mode. The ATPG patterns required are stored in LDEC. A self-comparable LCOMP unit is applied to reduce the generation overhead of golden results and locate faults (details in Sec. IV-B).

Fig. 4 (bottom) shows waveforms of BIST control signals for a DCIM configured with 512 rows, 256 columns and 32

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:48 UTC from IEEE Xplore. Restrictions apply.

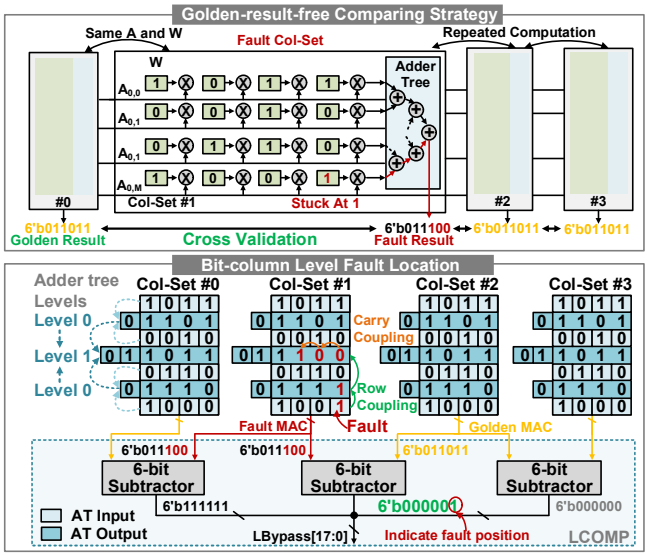


Fig. 5. The self-comparing and fault location strategy for DCIM.

input channels during the testing procedures of the March CCC algorithm. The BIST controller is activated by *CBIST_EN* signal and alters states based on test steps, generating control signals for MCNT and LCNT. Mode 1 (show in Fig. 4 bottom) demonstrates one testing procedure for memory and BM logic. As shown in the waveform in green, 512 writes in ascending address are performed with write enable (*WEN*) set to 1. MCNT counts test address (TADDR) in ascending order while MDEC decodes test weight (TW) to all 1s. For DCIM, computing input and write/read operations can be executed simultaneously. Therefore, computing enable (CEN) and read enable (REN) are raised with test activation (TA) equals to all 1s. This procedure corresponds to step 2 in Algorithm 1, detecting the memory faults and CIW faults in BM logic. After each read, MCOMP compares the weight read from DCIM with expected output (1s), asserting BIST_GO when they match. Other steps (step 1, 3-5) in March CCC algorithm can be performed similarly according to testing flow.

Mode 2 tests the adder tree with ATPG patterns. For a DCIM with 32 input channel, 32 writes are performed before computation (*WEN*=1) to load all weights to SRAMs in every ATPG pattern. The activation A is input to DCIM with *CEN* pulls up. The computation results are validated in the LCOMP unit, which outputs the LBypass signals for fault diagnosis. The test procedure finishes when ATPG patterns are exhausted. Finally, if the BIST procedure is completed without any faults, the signals BIST_DONE and BIST_GO are asserted to indicate successful execution. The number of additional logic testing vectors is correlated to the target fault coverage. Unlike MDEC, LDEC retrieves test vectors ((W,A)) from pattern buffer, typically implemented using ROM or SRAM. An improvement of fault coverage from 95% to 99% could lead to almost 5× increase of required test patterns, resulting in significant storage overhead (as indicated in Fig. 10). Therefore, a coverage-directed testing algorithm should be developed for high coverage with limited test patterns.

B. Self-Comparable Strategy

The logic test patterns of DCIM require significantly fewer patterns than conventional digital circuits, owing to its regular and repetitive adder tree architecture, thereby enabling efficient on-chip BIST implementation. To avoid golden MAC result generation with excessive overhead, a column-level self-comparable strategy is developed to design the LCOMP unit to validate the DUT MAC result. Due to the regularity, the adder trees in DCIM array can be partitioned into multiple same blocks (Col-Set in Fig. 5 top). Then, each block is tested separately with same ATPG pattens. As shown in Fig. 5 top, our strategy employs the same weights and activations for each Col-Set and compares the partial summations (PSUM) among adjacent Col-Sets, i.e. each adjacent column serves as a golden result for the other. Repeated computation is performed in each Col-Set to detect the faults in adder tree. The adjacent comparison generate N-1 validation result (True/False) for N Col-Sets, which indicates the location and number of fault Col-Sets. To further locate the fault bits, a subtractor is used to calculate the difference between two MAC results. Within the small area of a Col-Set, the probability of existing multiple faults is significantly low and single defect covers most of the cases. Therefore, single fault assumption is adopted within a Col-Set in the following discussion to simplify the analysis of bit-level fault location strategies. The difference between the correct MAC value and error MAC value can only be 2^n when one fault exists in DCIM macro, n indicates the bit position at which the fault occurs. Based on above analysis, we implement a self-comparable LCOMP to locate faults at bit-column level with low overhead.

Fig. 5 illustrates the LCOMP module with an example of accumulation for four 4-bit data. Every four columns of DCIM, i.e. a Col-Set, represents a 4-bit weight to compute with bit serial activations. The DCIM array is composed of repeating Col-Sets, allowing for the generation of test patterns for just one Col-Set. Each Col-Set adds the same four addends, while Col-Set #0 and Col-Set #2 generate golden MAC for Col-Set #1. To locate fault, a subtractor is used as comparator between Col-Sets. As displayed in Fig. 5, at Col-Set #1, a SAF happens at the lowest bit of the fourth addend. The 6-bit subtractor generates the difference of MAC values for adjacent Col-Sets. The second subtractor outputs the difference between fault MAC (6'b011100) and golden MAC (6'b011011) as 6'b000001, indicating a fault at the lowest bit-column in adder tree. The fault index (LBypass) will be sent to diagnose unit after comparison for future fault repair.

V. CIMTester: Automated CIM BIST Compiler

A. CIMTester Overview

The overview of CIMTester is illustrated in Fig. 6. It's a compilation framework that simulates faults in DCIM and generates CIM BIST algorithms and BIST circuits based on user specifications for fault specifications, architecture parameters, hardware constraints. CIMTester comprises a fault simulator, a template-based BIST generator and an exploration loop to

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:48 UTC from IEEE Xplore. Restrictions apply.

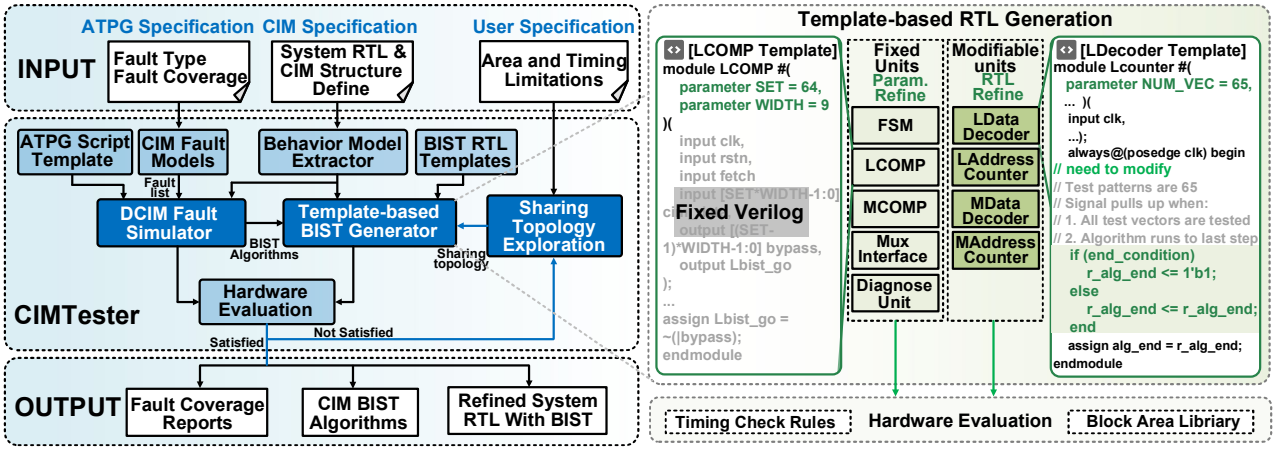


Fig. 6. The overview of CIMTester compilation framework.

investigate the sharing topology of BIST architecture. Before BIST circuit generation, the fault simulation is performed with the user specification for fault type and target fault coverage. The ATPG scripts templates are modified according to current CIM structure to generate ATPG patterns for adder tree with commercial ATPG tools. The fault simulator adopts the DCIM fault models discussed in Sec. III and reports the fault coverage after simulate all test vectors for every fault in fault list. The CIM BIST algorithm is returned when target fault coverage is achieved. After fault simulation, BIST compiler generates the BIST circuits based on proposed BIST structure detailed in Sec. IV. System RTL and CIM structure parameters are defined by users to describe the target single or multiple DCIM system. The macro structure definition should include macro rows, columns, input channels and supported weight & input precisions of all the CIM macros used in system RTL. Behavior models of CIM macros are better provided for ATPG and BIST generation processes. CIMTester also provides behavior model extractor to extract behavior model of CIM macros according to macro definition.

BIST compiler generates BIST RTL by modifying BIST RTL templates according to the DCIM behavior models and the CIM BIST algorithm. As shown in Fig. 6 right, the RTL template is split into two major parts: fixed components and modifiable units. The fixed part, such as interface multiplexers and FSM controllers, is implemented with optimized RTL. Their modification only involves parameter refinement for adaption to different DCIM macros. Meanwhile, the modifiable portions, such as LDEC, MDEC, LCNT and MCNT, which are related to the implementation of CIM BIST algorithms and DCIM structure, require modifications on function blocks in RTL. Hardware constraints as area and timing are also defined by users to satisfy the metric requirements of target system. The constraints can be formulated as an optimization problem of sharing topology exploration of multiple sets of BIST circuits for DCIM macros at system level with an upper limit for area and test time. CIM BIST specified timing check rules are applied to evaluate the test time of target system and block area is evaluated with block area library.

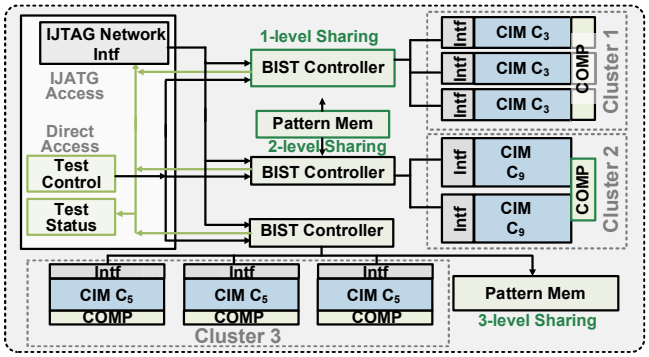


Fig. 7. The BIST architecture and sharing strategies of BIST modules.

B. System Level BIST Architecture Exploration

Since the DCIM is used for data-intensive computations, the number of DCIM blocks in a chip is large. As a result, the number of bits of computing inputs, computing outputs, and read/write IOs is huge, which cause significantly high area overhead of BIST circuit. Testing the DCIM block by block consumes extremely long test time and is different from the normal operation condition. Therefore, reasonable sharing strategies of BIST modules between macros should be customized case by case to balance the test parallelism and area overhead. For easier exploration and comparison, we design and generate the BIST circuits of a list of DCIM macros of different configurations as listed in Table. II. Fig. 7 shows a system with 8 DCIM macros (design A indicated in Table. III). The system level BIST architecture usually shares BIST modules to reduce area overhead. There are three levels of module sharing to explore in a multi-CIM BIST system, which is the sharing of BIST controller, test pattern memory and comparator (COMP). BIST controller and COMP can be shared in a CIM cluster gathering same type of CIM (eg. cluster 1 and 2 in Fig. 7). And pattern memory can be shared between clusters.

Based on these CIM macros, we demonstrate three levels of sharing strategies of 4 CIM system designs (listed as A, B, C and D in Table. III). Design A includes 3 CIM clusters, which contains 3 macros of type C_3 (3-C_3), 3 macros of type C_5 (3-C_5) and 2 macros of type C_9 (2-C_9) (as shown in Fig.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:48 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE BIST OVERHEAD OF DIFFERENT DCIM MACROS.

Category	Index	Size (Row×Col)	Input Channel	Adder Sets	Weight Format	BIST Area Overhead
Different Precision and Adder Sets	C_1	512×256	32	16	INT16	0.752%
	C_2		32	32	INT8	1.475%
	C_3		32	64	INT4	2.432%
	C_4		64	64	INT4	2.070%
	C_5		128	64	INT4	1.502%
Different Memory	C_6	1024×256	32	64	INT4	1.278%
	C_7	2048×256	32	64	INT4	0.657%
Different Memory and Adder Sets	C_8	512×512	32	128	INT4	2.435%
	C_9	512×1024	32	256	INT4	2.367%

TABLE III
THE BIST TOPOLOGY EXPLORATION OF EXAMPLE DESIGN CASES.

Design	System Config.	Sharing Strategy	Area Overhead	Test Time
A	3-C_3-3-C_5-2-C_9	Ctrl	4.9% (-)	11.2N[a] (20N[b])
B	9-C_3-9-C_5-9-C_9	Ctrl+PMEM	3.27% (-)	13.52N (20N)
B	9-C_3-9-C_5-9-C_9	Ctrl+PMEM+COMP	0.66% (2%[b])	35.06N (-)
C	8-C_3-2-C_5-1-C_9	Ctrl+PMEM+COMP	0.90% (-)	13.34N (20N)
D	4-C_3-4-C_5-4-C_9	Ctrl+PMEM	3.46% (-)	8.16N (20N)
D	4-C_3-4-C_5-4-C_9	Ctrl+PMEM+COMP	1.29% (4%)	26.12N (30N)

[a]N stands for the test time for one C_3 macro
[b]Target area or test time limitation set by user.

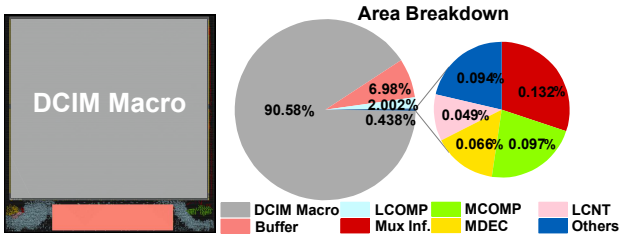
7). The configuration of design B, C and D are indicated as 9-C_3-9-C_5-9-C_9, 8-C_3-2-C_5-1-C_9 and 4-C_3-4-C_5-4-C_9. As listed in Table. III, the topology exploration has been performed under different constraints of test time and area overhead. Under the test time limitation of 20N, design A can achieve a 11.2N test time with an overhead of 4.9% using only the sharing of controller in cluster (1-level sharing). For design B, an extremely low area overhead of 0.66% has been achieved with 3 levels of sharing while the test time increased from 3.34N (full test parallelism) to 35.06N. The BIST generation of design D has constraints on both test time and area, which achieves a test time of 26.12N and an area overhead of 1.29%. Table. III shows part of generation results that satisfy the user specification on the 4 design cases. We will provide more quantitative studies and comparison in Sec. VI-D.

VI. EXPLORATIONS AND EVALUATIONS

A. Implementation and Comparison

In this paper, we implemented the BIST circuit for a series of CIM macros using TSMC 22nm technology, considering various configurations (input channels and weight precision, eg. 32c-16b) and sizes (rows by columns, eg. 512x256). Fig. 8 shows the layout and area breakdown of the BIST circuit for a 512x256 DCIM macro. The pattern buffer occupies 6.98% of the total area, which can be shared across multiple DCIM macros on the chip to significantly reduce its proportion. Apart from DCIM macro and buffer, the BIST circuit occupies only 2.44% of total area. Among the BIST circuits, LCOMP consumes most of the area with an area proportion of 2.002% (highlighted in blue in layout and area breakdown), which consists of multi-bit subtractors.

Fig. 8 (bottom) compares the CIM-BIST implementations of this work with those of [15] and [16]. Our approach improves logic fault coverage by covering coupling faults (CIWF and



	[16]	[15]	This Work
CIM Type	Analog CIM	Digital CIM	Digital CIM
Macro Size	16x192	32x128	512x256 (32 input channels)
Area Overhead	0.92%	3.30%	2.44%
Weight Precision	INT8	INT4	INT4
MBIST Algorithm	Random	March X	March C- (can be replaced)
Logic Fault Types	-	SAF	SAF, TF, CIWF, COWF, SOF, etc.
Overall Fault Coverage	-	61%	99.48%
Fault Location	☹	☹	Bit-Column Level

[1]The fault coverage (not provided in [15]) is estimated with our CIM fault model.
[2]The test pattern is generated randomly and fault coverage is not provided in [16]

Fig. 8. The BIST implementation and comparison table.

COWF) between BM logic and SRAM cells and also logic faults in adder tree, achieving the highest coverage of 99.48% among all BIST implementations. The March C- algorithm is selected for MBIST, providing full coverage for SAFs and CFs in memory, and additionally offering partial coverage for CF compared to March X in [15]. In terms of area overhead, the BIST area ratio is smaller than that of [16], even with a DCIM macro 32 times larger. Although [15] shows a smaller area ratio with random testing strategy, the fault coverages for memory and computation logic are sacrificed. Regarding fault localization, our method detects faults at the column level, identifying the fault bits in the PSUMs for subsequent diagnosing, whereas the other approaches lack fault localization capabilities.

B. BIST Circuit Generation and Overhead

To evaluate the generation capability of our template-based BIST compiler, several BIST circuits are generated and implemented for DCIM macros using TSMC 22nm technology with a target fault coverage of 99%. As displayed in Fig. 9, DCIM macros are divided into two sets with the 512x256 macro with 32 input channels and 4-bit precision (32c-4b) as baseline.

The first set includes 512x256 macros that differ in input channels and weight precisions. As weight precision gets lower, BIST area overhead increases because more output channels lead to larger comparing logic for PSUMs. When input channels increase, the data precision of each output channel is higher, resulting in more BIST area overhead of more gates in LCOMP for extra bits. The second set consists of 32c-4b macros with different rows and columns. As the result shows, increasing row number results in negligible changes to BIST area, while the area ratio decreases as the total area of the DCIM macro increases. More columns in the DCIM macro require more self-compare logic due to the increased number of output PSUMs. However, the area ratio increases only marginally, as the overhead from PSUM comparison logic grows at the same rate as the DCIM macro area. Overall, the BIST area ratio for different DCIM macros ranges from 0.66% to 2.44% with target fault coverage of 99%.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:48 UTC from IEEE Xplore. Restrictions apply.

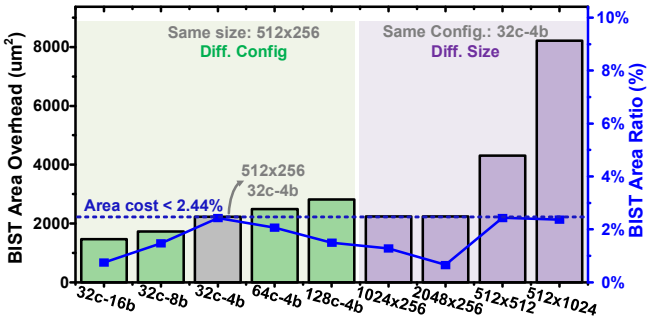


Fig. 9. The area overhead and area ratio of generated BIST circuit for various CIM macros.

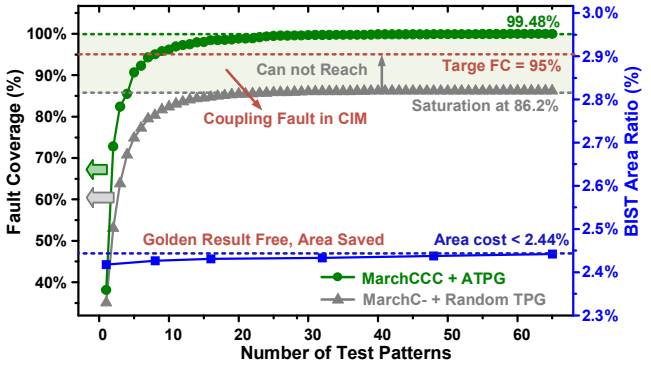


Fig. 10. The fault coverage and area ratio of BIST circuit.

C. Fault Coverage of Generated Patterns

Fig. 10 illustrates the fault coverage (FC) and BIST area overhead as the number of test patterns increases. The FC of March CCC algorithms is simulated based on the proposed CIM fault models with our fault simulator. And random TPG serves as the baseline, as it is commonly employed in LBIST. At early stage, fault coverage of March CCC grows rapidly at a similar rate to that of March C- algorithm with random TPG, because memory faults are detected with the same March C- algorithm. The FC of March CCC grows faster than the baseline because it detects CIW faults and COW faults in BM logic while reading/writing memory cells. However, random TPG soon saturates at 86.2% coverage, indicating the remaining logic faults are random-resistant. In contrast, March CCC algorithm with ATPG pattern for adder tree can outperform random TPG with additional coverage of computation coupling faults. By exploring more undetected paths, March CCC can achieve an overall fault coverage of 99.48% for both Memory faults and computing faults such as SAF, TF, CIWF, COWF, SOF and etc.

Due to the self-compare strategy of our BIST circuits eliminating area overhead of golden result storage, the BIST area ratio shows negligible growth as the number of test patterns increases. Results show that the area overhead of BIST is only 2.44% for a 4-bit 32-channel 512x256 DCIM macro while reaching a fault coverage of 99.48%.

D. The Sharing Topology Exploration

Fig. 11 details the exploration space of 4 design cases listed in Table. III under different user specification. The baseline indicates the shortest test time and largest area overhead when

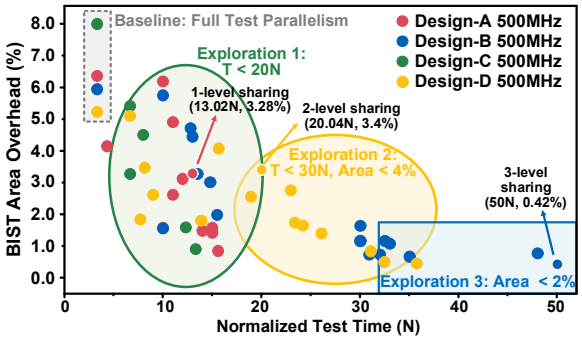


Fig. 11. The design space of system level CIM BIST architecture under three design cases.

no module sharing is applied on BIST architecture of CIM system. The first exploration limits the test time under 20N, where N is the test time of one C_3 macro. The exploration space is indicated in green and the BIST generation results for design A, B, C, D are plotted in different colors. The area overhead can range from 6.19% to 0.84% with only limitations on test time. 1-level or 2-level sharing strategy is preferred in exploration 1 because higher test parallelism is required to achieve shorter test time. Exploration 3 limits the area to less than 2% for design B and D, demonstrating the testing case when area is very limited for BIST circuit. As shown in exploration space in blue, strict area limitation leads to increased test time. 3-level sharing strategy is preferred in this situation and most of the test time are consumed on waiting for the shared modules. A extremely low BIST area overhead of 0.42% is achieved for design B under the area constraint while the test time increased over 10× than the baseline. More balanced BIST architectures can be generated when provides reasonable constraints on both area and test time. As shown in Fig. 11, timing and area constraint of 30N and 4% are applied on BIST generation for design B and D, indicating with yellow circle. This exploration space demonstrates moderate area overhead and test time. A 2-level sharing strategy is applied on design D, achieving an area overhead of 3.4% and a test time of 20.04N. The required BIST architecture of multi-CIM system varies case by case and user by user. Therefore, an cost-effective BIST compiler that can generate satisfying BIST structure is in urgent need.

VII. Conculsion

This paper introduces CIMTester, a template-based BIST compiler that generates cost-effective CIM BIST circuit for CIM system through a self-comparison strategy. A golden-result-free BIST structure and a bit-column level fault location method are proposed to test DCIM macros with significantly small area overhead. The implementation result demonstrates a significantly high fault coverage of 99.48% with less than 2.44% area overhead with TSMC 22nm technology. CIM specified fault simulator is integrated in CIMTester to provide fault simulation and coverage reports for DCIM system.

Acknowledgement

This work is supported by NSFC No. 92164301, 92464301, 62225401, U24A20287.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:48 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] H. Fujiwara, H. Mori, W.-C. Zhao, M.-C. Chuang, R. Naous, C.-K. Chuang, T. Hashizume, D. Sun, C.-F. Lee, K. Akarvardar, S. Adham, T.-L. Chou, M. E. Sinangil, Y. Wang, Y.-D. Chih, Y.-H. Chen, H.-J. Liao, and T.-Y. J. Chang, "A 5-nm 254-TOPS/W 221-TOPS/mm ^{2} Fully-Digital Computing-in-Memory Macro Supporting Wide-Range Dynamic-Voltage-Frequency Scaling and Simultaneous MAC and Write Operations," in *2022 IEEE International Solid- State Circuits Conference (ISSCC)*, (San Francisco, CA, USA), pp. 1–3, IEEE, Feb. 2022.

[2] H. Mori, W.-C. Zhao, C.-E. Lee, C.-F. Lee, Y.-H. Hsu, C.-K. Chuang, T. Hashizume, H.-C. Tung, Y.-Y. Liu, S.-R. Wu, K. Akarvardar, T.-L. Chou, H. Fujiwara, Y. Wang, Y.-D. Chih, Y.-H. Chen, H.-J. Liao, and T.-Y. J. Chang, "A 4nm 6163-TOPS/W/b \mathbf4790-TOPS/mm^2/b SRAM Based Digital-Computing-in-Memory Macro Supporting Bit-Width Flexibility and Simultaneous MAC and Weight Update," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pp. 132–134, Feb. 2023. ISSN: 2376-8606.

[3] M.-E. Shih, S.-W. Hsieh, P.-Y. Tsai, M.-H. Lin, P.-K. Tsung, E.-J. Chang, J. Liang, S.-H. Chang, C.-L. Huang, Y.-Y. Nian, Z. Wan, S. Kumar, C.-X. Xue, G. Jedhe, H. Fujiwara, H. Mori, C.-W. Chen, P.-H. Huang, C.-F. Juan, C.-Y. Chen, T.-Y. Lin, C. Wang, C.-C. Chen, and K. Jou, "20.1 NVE: A 3nm 23.2TOPS/W 12b-Digital-CIM-Based Neural Engine for High-Resolution Visual-Quality Enhancement on Smart Devices," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, (San Francisco, CA, USA), pp. 360–362, IEEE, Feb. 2024.

[4] F. Tu, Z. Wu, Y. Wang, W. Wu, L. Liu, Y. Hu, S. Wei, and S. Yin, "MulTCIM: Digital Computing-in-Memory-Based Multimodal Transformer Accelerator With Attention-Token-Bit Hybrid Sparsity," *IEEE Journal of Solid-State Circuits*, pp. 1–12, 2023.

[5] R. Guo, L. Wang, X. Chen, H. Sun, Z. Yue, Y. Qin, H. Han, Y. Wang, F. Tu, S. Wei, Y. Hu, and S. Yin, "20.2 A 28nm 74.34TFLOPS/W BF16 Heterogenous CIM-Based Accelerator Exploiting Denoising-Similarity for Diffusion Models," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67, pp. 362–364, Feb. 2024. ISSN: 2376-8606.

[6] X. Yu, T. Ni, X. Sheng, Y. Pan, L. He, and L. Zhao, "AESHA: Accelerating Eigen-decomposition-based Sparse Transformer with Hybrid RRAM-SRAM Architecture," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, (Newark Liberty International Airport Marriott New York NY USA), pp. 1–9, ACM, Oct. 2024.

[7] J. Park, K. Lee, and J. Park, "TP-DCIM: Transposable Digital SRAM CIM Architecture for Energy-Efficient and High Throughput Transformer Acceleration," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, (Newark Liberty International Airport Marriott New York NY USA), pp. 1–8, ACM, Oct. 2024.

[8] A. Z. Jidin, R. Hussin, L. W. Fook, and M. S. Mispan, "A review paper on memory fault models and test algorithms," *Bulletin of Electrical Engineering and Informatics*, vol. 10, pp. 3083–3093, Dec. 2021.

[9] "Memory Testing and Built-In Self-Test," in *VLSI Test Principles and Architectures*, pp. 461–515, Morgan Kaufmann, Jan. 2006.

[10] T.-L. Tsai, J.-F. Li, C.-L. Hsu, and C.-T. Sun, "Testing of In-Memory-Computing 8T SRAMs," in *2019 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, (Noordwijk, Netherlands), pp. 1–4, IEEE, Oct. 2019.

[11] M. Fieback, S. Nagarajan, R. Bishnoi, M. Tahoori, M. Taouil, and S. Hamdioui, "Testing Scouting Logic-Based Computation-in-Memory Architectures," in *2020 IEEE European Test Symposium (ETS)*, (Tallinn, Estonia), pp. 1–6, IEEE, May 2020.

[12] J.-F. Li, T.-L. Tsai, C.-L. Hsu, and C.-T. Sun, "Testing of Configurable 8T SRAMs for In-Memory Computing," in *2020 IEEE 29th Asian Test Symposium (ATS)*, pp. 1–5, Nov. 2020. ISSN: 2377-5386.

[13] J.-F. Li, "Testing of Computing-In Memories: Faults, Test Algorithms, and Design-for-Testability," in *2023 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, pp. 1–6, Oct. 2023. ISSN: 2765-933X.

[14] J.-F. Li, "Testing and Reliability of Computing-In Memories: Solutions and Challenges," in *2022 IEEE International Test Conference in Asia (ITC-Asia)*, (Taipei, Taiwan), pp. 55–60, IEEE, Aug. 2022.

[15] H. Zhang, H. Zhu, S. He, M. Li, C. Wang, X. Xiong, H. Tian, X. Zeng, and C. Chen, "ARCTIC: Agile and Robust Compute-In-Memory Compiler with Parameterized INT/FP Precision and Built-In Self Test," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–6, Mar. 2024. ISSN: 1558-1101.

[16] Y.-C. Tsai, W.-C. Ting, C.-C. Wang, C.-C. Chang, and R.-S. Liu, "Built-in Self-Test and Built-in Self-Repair Strategies Without Golden Signature for Computing in Memory," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, (Antwerp, Belgium), pp. 1–6, IEEE, Apr. 2023.

[17] P. Muthukrishnan and S. Sathasivam, "A Technical Survey on Delay Defects in Nanoscale Digital VLSI Circuits," *Applied Sciences*, vol. 12, p. 9103, Jan. 2022. Number: 18 Publisher: Multidisciplinary Digital Publishing Institute.

[18] A. Ahmad and S. S. Al Busaidi, "Chapter 17 - Fault diagnosis and fault tolerance," in *System Assurances* (P. Johri, A. Anand, J. Vain, J. Singh, and M. Quasim, eds.), Emerging Methodologies and Applications in Modelling, pp. 307–321, Academic Press, Jan. 2022.

[19] Y.-D. Chih, P.-H. Lee, H. Fujiwara, Y.-C. Shih, C.-F. Lee, R. Naous, Y.-L. Chen, C.-P. Lo, C.-H. Lu, H. Mori, W.-C. Zhao, D. Sun, M. E. Sinangil, Y.-H. Chen, T.-L. Chou, K. Akarvardar, H.-J. Liao, Y. Wang, M.-F. Chang, and T.-Y. J. Chang, "16.4 An 89TOPS/W and 16.3TOPS/mm2 All-Digital SRAM-Based Full-Precision Compute-In Memory Macro in 22nm for Machine-Learning Edge Applications," in *2021 IEEE International Solid- State Circuits Conference (ISSCC)*, vol. 64, pp. 252–254, Feb. 2021. ISSN: 2376-8606.

[20] M. Wu, W. Ren, P. Chen, W. Zhao, Y. Jing, J. Ru, Z. Wang, Y. Ma, R. Huang, T. Jia, and L. Ye, "S2D-CIM: A 22nm 128Kb Systolic Digital Compute-in-Memory Macro with Domino Data Path for Flexible Vector Operation and 2-D Weight Update in Edge AI Applications," in *2024 IEEE Custom Integrated Circuits Conference (CICC)*, pp. 1–2, Apr. 2024. ISSN: 2152-3630.

[21] J.-F. Li, "Special Session: Testing of Digital Computing-In Memories with MAC Function," in *2024 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, pp. 1–5, Oct. 2024. ISSN: 2765-933X.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:48 UTC from IEEE Xplore. Restrictions apply.